Programming Project #1
EGRE246
Simulating a Forest Fire

1 Overview

Write a C program to implement a cellular automaton simulation of fire spreading through a forest. The initial state of the forest will be inputted from a text file and you will then process the fire spreading through the forest until no more fire exists then outputting the final state of the forest after the fire.

2 Cellular Automata

A **cellular automaton** is a collection of cells on a grid (think of a two-dimensional space represented by a two-dimensional array in a program) of a specified shape that evolves through a number of discrete time steps according to a set of rules based on the states of neighboring cells. The rules are then applied iteratively for as many time steps as desired.

For example, consider this small 3-by-3 grid with each cell containing a blank or the letter **x**:

Generation #1

	1	2	3
1			
2	**x**	**x**	
3	**x**		**x**

Now we wish to look at each cell in turn and apply the following rules:

if a cell has exactly 2 neighbor cells (don't count the cell being considered!) with an **x** *then*
    an **x** will remain or be created in that cell in the next generation (a new 3-by-3 grid)
else
    the same cell in the next generation will contain a blank

So applying these rules to each cell one by one in the example above yields the following grid (here we are considering diagonal cells as neighbors):

Generation #2

	1	2	3
1	**x**	**x**	
2	**x**		**x**
3	**x**		

Our automaton would continue by applying the same rules to this new generation to produce the next one:

Generation #3

	1	2	3
1	**X**		**X**
2			
3			

Cellular automatons usually continue like this producing generation after generation until some defined stop state is reached. Note that in the above example the next generation (generation #4) would contain all blanks.

Cellular automatons can be very useful in modeling stochastic systems across many fields. `https://en.wikipedia.org/wiki/Cellular_automaton`

3 Fire in Our Forest

If a cell in our forest is on fire, then that fire spreads to all neighboring cells in one generation with some probability determined by factors such as wind, precipitation falling, dryness of leaves and underbrush, etc. While it would be possible in a more complicated simulation to model all of this uncertainty, we are going to greatly simplify things by providing a single probability that at each iteration fire will spread from one burning cell to another. We will specify this probability, as a percentage from 0% to 100% in our input file. In other words if there is a 100% probability that fire will spread it then will always catch a neighboring cell on fire, or if the probability is 50% then only half of the time will a neighboring cell actually catch on fire in one generation, etc. We will use a random number generator to determine the probability of fire spreading; more on this later. To simplify matters somewhat we will not be considering diagonal cells as neighbors; therefore example cell **(r,c)** below has only 4 neighbors:

	c-1	c	c+1
r-1		(r-1,c)	
r	(r,c-1)	**(r,c)**	(r,c+1)
r+1		(r+1,c)	

Fire completely burns a tree in one generation leaving behind an empty cell. Also you are to completely scan your forest moving left-to-right, top-to-bottom looking at *every* cell. Generate a random number only if the current cell is a tree and if it has a neighbor on fire. (There are several ways to process the cells in a generation; however, only if all solutions are done in this manner will each random number in the sequence be used in the same way for all of your programs therefore giving the same result for a given machine.)

4 Implementation

4.1 Representation of the Forest

You will represent your forest as an x-by-y two dimensional array of integers. Each cell in your array will contain one of 3 possible values: 0 means the cell is empty, 1 means it contains a tree, and 2 means fire. The initial state of your forest will be determined by input from a text file (explained below); assume a cell in the forest is empty if not specified. We will initialize the first and last rows plus the first and last columns (i.e. all the border squares) to empty (0) to make your program logic easier; think of it as a built-in fire barrier (fire will not be allowed to jump empty spaces). The largest forest we will consider is 100-by-100; therefore to represent a 100-by-100 forest you will need to create an array with at least 102 rows and 102 columns.

4.2 Program input

The initial state of the forest will be specified by input from a text file whose name will be given on the command line. The text file will have the following format; note that bold remarks below are for explanation and *not* part of the actual data in the file:

```
8       ← number of rows
20      ← number of columns
28      ← random number generator seed s (exit program with message if s < 0)
75      ← percentage probability p of fire burning a neighbor (exit program with message if p < 0)
T 5 8   ← place a tree at row 5, column 8
A       ← place a tree in all squares of the forest
F 3 4   ← row 3 column 4 is on fire
E 1 12  ← row 1 column 12 is empty
E 2 12  ← row 2 column 12 is empty
E 3 12  ← row 3 column 12 is empty
E 4 12  etc.
E 4 13
E 4 14
E 5 12
E 5 15
E 6 12
E 1 16
E 2 16
E 3 16
E 4 16
E 7 1
E 7 2
E 7 3
```

```
E 7 4
E 8 3
Q       ← quit inputting values i.e. the end of input
```

Note that spaces are required between values and all letter commands must be upper case. A latter command will essentially "overrule" a previous one, i.e. the sequence

```
T 6 7
E 6 7
F 6 7
```

will leave cell row 6 column 7 on fire.

Also note that this presents a rather tedious way to specific individual cells and there could be a more powerful way to specify trees, spaces, and fire. However implementing input is made much simpler by adopting this scheme. You may assume that the input data is correct (with the exceptions mentioned above); the result of providing erroneous input data to your program is undefined.

4.3 Printing the forest

You should print out your forest *exactly* (including the numbering of rows and columns plus table separators) in the manner show below with the character `F` representing fire, periods (`.`) representing empty space, and a `T` indicating a tree. Here is the initial representation of the forest using the data that was presented above:

```
          1         2
  12345678901234567890
 +--------------------+
1|TTTTTTTTTT.TTT.TTTT| 1
2|TTTTTTTTTT.TTT.TTTT| 2
3|TTTFTTTTTT.TTT.TTTT| 3
4|TTTTTTTTTT...T.TTTT| 4
5|TTTTTTTTTT.TT.TTTTT| 5
6|TTTTTTTTTT.TTTTTTTT| 6
7|....TTTTTTTTTTTTTTT| 7
8|TT.TTTTTTTTTTTTTTT| 8
 +--------------------+
  12345678901234567890
          1         2
```

4.4 Using random numbers to provide probability

After seeding the random number generator once (by using `srand()` and the inputted seed value; remember that a given random number generator will always produce the same sequence of values for any given seed), we will use a call to `rand()` which gives us a random integer value rn such that $0 <= rn <$ `RAND_MAX`. We wish to map these random values as floats into the range 0 to 1; here is how we would do that:

```
float rn = (float)rand()/(float)RAND_MAX;
```

Consider these values. If they are truly a random distribution between 0 and 1 then 25% of the time `rn` will be less than or equal to 0.25, 50% of the time `rn` $\leq$ 0.50, 75% of the time `rn` $\leq$ 0.75, etc. We can now use this to create random events with a certain probability of occurring (where `prob` below is a value between 0 and 1, i.e. the probability percentage inputted divided by 100.0):

if rn $\leq$ prob then
an event occurred

So applying this to our simulation we get the following logic to apply to each cell in our forest:

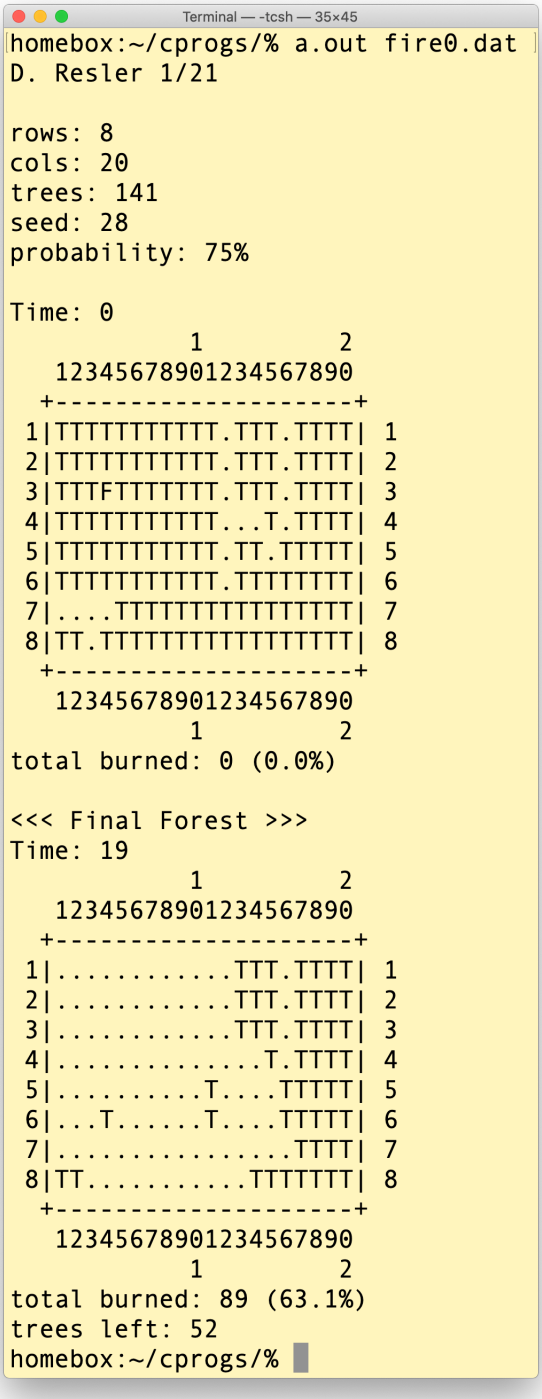
if cell c **is a tree and a neighbor is burning then**
  if a generated random number between 0.0 and 1.0 is less than `prob` **then**
    cell c **is on fire in the next generation**
  else
    cell c **remains a tree in the next generation**

Note that you will also have to decide in your forest traversal what to do for the cells that you encounter that are already on fire or empty.

5 Project Specification and Sample Run

Write a C program that inputs the initial state of the forest plus simulation parameters from an input file (in the format with the types of values given above) given on the command line and then outputs the initial generation (generation #0) with parameters and the final generation (defined as the first generation reached with no fire) plus some simulation statistics. Your code must produce output formatted exactly as the sample run below. You should subdivide your tasks into reasonable subprograms (functions) and you should not use any global variables (other than preprocessor definitions).

Example run of program (note that you will replace my name in your program with your own plus you will need to create your own test data file(s)) using the data file given in this handout:



Note that the C standard does not require that the function `rand()` will give the same sequence of values for the same seed across different machines! Therefore your results might and probably will be slightly different than mine given above even with

the same inputs.

6 Implementation Suggestions

Some general suggestions on how to approach this project.

1. Begin by getting your file input and the printing of your initial configuration correct for a variety of forest sizes.

2. Use the following general algorithm as a starting point for your program:

   initializations
   input data
   set the time
   output initial info
   while there is still fire in the forest do
   compute next generation
   increment time
   endwhile
   output final generation and info

3. Create a debug mode where you print out every generation that is created. When you do this you will need to pause the output after each generation; probably the easiest way to do this:

   ```
   void pause() {
     printf("<press return to continue>");
     getchar();
   }
   ```

   Do not forget to turn off the printing of every generation before you submit your project!

4. First implement your simulation without using probabilities, i.e. the probability will be 100%. It is much easier to see if your simulation is working correctly this way.

5. Use a small data set while developing your code – it's much easier to see if your code is working correctly with less data.

7 Documentation

- You need to work alone on this project!

- *All* projects this semester that have a `main` function should output (to standard output) the project number and author's name as the *first* line of your output.
- *Every* file turned in this semester should include a block of comments at the very beginning of the file that lists at least the project number and author's name.

8 Deliverables

Submit a single '`.c`' file containing your source code. Projects (i.e. your source code file(s)) this term should be submitted via Canvas.